DOCUMENTACIÓN TÉCNICA OFICIAL · VERSIÓN 2.0.0

Hoja de Ruta - Rol 1: Ingeniero de Backend Core, Motor P2P y Lógica de Datos

Construir una API robusta, atómica y determinista que gestione el ciclo de vida del cupón, garantice el cumplimiento del Price Collar, ejecute el matching P2P sin race conditions y audite los fondos de custodia.

Proyecto / Sistema	**Sistema de Mercado Comportamental y Trading de Activos (SMCTA v2)**
Fase / Área Normativa	**Lógica de Negocio, Base de Datos, Motor de Emparejamiento P2P y Escrow Ledger**
Autoría y Gobernanza	**Equipo de Ingeniería de Software — Proyecto SMCTA**
Fecha de Emisión	**Noviembre 2026**

1. Documentación Base de Contexto (Lectura Obligatoria para tu IA)

Antes de escribir código, proporciona a tu IA asistente los siguientes documentos ubicados en el repositorio:

1. `05_Fase_Gestion_de_Proyecto_y_Roadmap/Hojas_de_Ruta_Distribuidas/00_Protocolo_de_Alineacion_y_Contratos_Compartidos.md` (Contrato maestro de DTOs y endpoints).
2. `03_Fase_Arquitectura_y_Diseño_de_Software/02_Diseno_de_Modelo_de_Datos_y_Entidades.md` (DDL de PostgreSQL, llaves foráneas y tipos).
3. `02_Fase_Especificacion_de_Requisitos_y_Negocio/02_Especificacion_de_Reglas_de_Negocio_y_Motor_P2P.md` (Máquina de estados y reglas RN-01 a RN-04).
4. `01_Fase_Estrategia_y_Rediseño_de_Modelo/02_Propuesta_Redisenio_Arquitectura_P2P_v2.md` (Flujos de información y mitigación de riesgos).
5. `02_Fase_Especificacion_de_Requisitos_y_Negocio/03_Matriz_de_Comisiones_y_Monetizacion_WhiteLabel.xlsx` (Tabla de parámetros por defecto y simulaciones financieras).

2. Stack Tecnológico Recomendado

- **Runtime & Lenguaje:** Node.js (v20+ LTS) con TypeScript en modo estricto (`strict: true`).
- **Framework Web:** Fastify o Express con validación de esquemas mediante `Zod` .
- **Base de Datos:** PostgreSQL 15+ administrado con `Prisma ORM` o `Kysely` (evitar ORMs con sobrecarga en transacciones atómicas).
- **Caché y Concurrencia:** Redis (mediante `ioredis`) para locks distribuidos (*Redlock* o semáforos de clave única).
- **Precisión Numérica:** `decimal.js` (CRÍTICO: nunca operar divisas ni porcentajes con `Number` flotante nativo de JavaScript).
- **Testing:** Vitest o Jest + Supertest + Testcontainers (para levantar Postgres y Redis reales en tests).

3. Plan de Trabajo por Sprints

Sprint 6: Fundamentos de Datos, Checkout Primario y QR Criptográfico

- **Tarea 1.1 (TASK-001):** Inicializar proyecto TypeScript e implementar el DDL completo en migraciones reproducibles (`tenants` , `users` , `coupons` , `p2p_orders` , `escrow_ledgers`).
- **Tarea 1.2 (TASK-001 / US-01):** Crear servicio y endpoint `POST /api/v1/checkout/primary` .
- Crear registro de cupón en estado `EN_WALLET` .
- Crear asiento en `escrow_ledgers` con `transaction_type = 'CHECKOUT'` y registrar el valor nominal exacto.
- **Tarea 1.3 (TASK-003 / US-02):** Implementar servicio de generación de tokens QR criptográficos (`GET /api/v1/coupons/:id/qr-token`).
- Generar nonce aleatorio y timestamp actual.
- Firmar con HMAC-SHA256 utilizando la clave privada del tenant.
- Verificar que el token expire a los 30 segundos exactos.
- Si el cupón tiene estado `PUBLICADO_P2P` , bloquear emisión y retornar HTTP 409 (`COUPON_NOT_IN_WALLET`).

Sprint 7: Motor de Órdenes P2P, Price Collar y Prevención de Concurrencia

- **Tarea 1.4 (TASK-004 / US-03):** Crear endpoint `POST /api/v1/p2p/orders` con validador de banda de precios (RN-02).
- Consultar parámetros del tenant (`price_floor_pct` , `price_ceiling_pct` , `nominal_price`).
- Calcular rango: $P_{min} = \text{nominal} \times (\text{price_floor_pct} / 100)$, $P_{max} = \text{nominal} \times (\text{price_ceiling_pct} / 100)$.
- Si `asking_price` $< P_{min}$ o $> P_{max}$, rechazar con código **HTTP 422** y error `PRICE_COLLAR_VIOLATION` .
- Validar límite de 5 publicaciones activas por usuario (`maxDailyResalesPerUser`).
- Actualizar estado atómico del cupón a `PUBLICADO_P2P` .
- **Tarea 1.5 (TASK-005 / US-03):** Implementar Lock Concurrente en Redis para prevenir *Double Spending*:
- Adquirir lock sobre la clave `lock:coupon:{coupon_id}` con TTL de 5 segundos antes de publicar o vender.

- **Tarea 1.6 (TASK-006 / US-04):** Construir el **Matching Engine P2P y Split de Comisiones**:
- Endpoint `POST /api/v1/p2p/orders/:id/buy`.
- Transacción PostgreSQL con nivel de aislamiento `SERIALIZABLE`.
- Deducir comisiones según parámetros del tenant:
- Fee Vendedor = $P_{asking} \times (seller_take_rate / 100)$
- Fee Comprador = $P_{asking} \times (buyer_take_rate / 100)$
- Neto Vendedor = P_{asking} - Fee Vendedor
- Total Pagado por Comprador = P_{asking} + Fee Comprador
- Reparto Plataforma = (Fee Vendedor + Fee Comprador) × (platformRevenueSharePct / 100)
- Transferir propiedad del cupón al nuevo comprador (`current_owner_id = buyer_id`) y regresar su estado a `EN_WALLET`.
- Insertar registros en `escrow_ledgers` tipo `P2P_SPLIT`.

Sprint 8: Liquidación TPV, Webhooks de Escrow y Auditoría

- **Tarea 1.7 (TASK-008 / US-06):** Endpoint de validación TPV `POST /api/v1/tpv/validate-qr`:
- Validar firma HMAC y que el timestamp no supere 30 segundos.
- Verificar que el estado sea `EN_WALLET`.
- Mutar estado atómicamente a `CANJEADO`.
- Emitir evento/webhook interno de liberación de Escrow (`REDEEM_RELEASE`).
- **Tarea 1.8 (TASK-006):** Módulo de conciliación y auditoría de balance:
- Crear endpoint interno `GET /api/v1/escrow/audit/:tenant_id` que compare el total de fondos retenidos en `escrow_ledgers` con los cupones vivos (`EN_WALLET` + `PUBLICADO_P2P`). La diferencia debe ser exactamente 0.

Sprint 9: Pruebas de Carga, Concurrencia y Optimización

- **Tarea 1.9 (QA-TASK-005):** Crear suite de tests de estrés concurrente: 50 compradores intentando adquirir la misma orden P2P al mismo milisegundo. Solo 1 debe ganar (HTTP 200) y los otros 49 deben recibir HTTP 423 (`P2P_ORDER_LOCKED`) o HTTP 409 sin corromper la base de datos.
- **Tarea 1.10:** Creación de imagen Docker para el Backend Core y documentación Swagger/OpenAPI.

---

4. Prompt Maestro para Iniciar con tu IA Asistente

Copia y pega el siguiente prompt en tu IA (Claude, ChatGPT, Cursor, etc.):

```
TEXT
1 Actúa como Ingeniero Principal de Backend y Arquitecto de Sistemas Financieros. Tu responsabilidad es implementar el
Backend Core y el Motor de Órdenes P2P para el proyecto SMCTA (Sistema de Mercado Comportamental y Trading de Activos).
2
3 CONTEXTO Y REGLAS OBLIGATORIAS:
4 1. Lee y respeta los contratos de '00_Protocolo_de_Alineacion_y_Contratos_Compartidos.md'. No alteres nombres de
campos, tipos ni códigos HTTP (ej. HTTP 422 para Price Collar).
5 2. Lee el DDL de '03_Fase_Arquitectura_y_Diseño_de_Software/02_Diseno_de_Modelo_de_Datos_y_Entidades.md' y las reglas
de negocio de '02_Fase_Especificacion_de_Requisitos_y_Negocio/02_Especificacion_de_Reglas_de_Negocio_y_Motor_P2P.md'.
6 3. Toda la aritmética monetaria debe usar 'decimal.js'. Jamás uses floats.
7 4. Para evitar condiciones de carrera en el matching P2P, implementa transacciones PostgreSQL SERIALIZABLE y Redis
distributed lock sobre 'lock:coupon:{id}'.
8 5. La máquina de estados del cupón es estricta: EMITIDO → EN_WALLET → PUBLICADO_P2P → CANJEADO / VENCIDO.
9
10 OBJETIVO INICIAL:
11 Comencemos por el Sprint 6. Genera el setup del proyecto con TypeScript, el esquema de Prisma/SQL con migraciones, y
los endpoints de Checkout Primario con asiento en 'escrow_ledgers' y generación de QR rotativo HMAC-SHA256 de 30
segundos. Proporciona código completo, modular y con pruebas unitarias.
```